

Name : Karmadipsinh Jadeja
Sub : AI
Enroll no : 92301733023

Exp-14: Sudoku Implementation

Code :

```
# Function to check if it is safe to place num at mat[row][col]
```

```
def isSafe(mat, row, col, num):
```

```
    # Check if num exists in the row
```

```
    for x in range(9):
```

```
        if mat[row][x] == num:
```

```
            return False
```

```
    # Check if num exists in the col
```

```
    for x in range(9):
```

```
        if mat[x][col] == num:
```

```
            return False
```

```
    # Check if num exists in the 3x3 sub-matrix
```

```
    startRow = row - (row % 3)
```

```
    startCol = col - (col % 3)
```

```
    for i in range(3):
```

```
        for j in range(3):
```

```
            if mat[i + startRow][j + startCol] == num:
```

```
                return False
```

```
return True
```

```
# Function to solve the Sudoku problem
```

```
def solveSudokuRec(mat, row, col):
```

```
    # base case: Reached nth column of the last row
```

```
    if row == 8 and col == 9:
```

```
        return True
```

```
    # If last column of the row go to the next row
```

```
    if col == 9:
```

```
        row += 1
```

```
        col = 0
```

```
    # If cell is already occupied then move forward
```

```
    if mat[row][col] != 0:
```

```
        return solveSudokuRec(mat, row, col + 1)
```

```
    for num in range(1, 10):
```

```
        # If it is safe to place num at current position
```

```
        if isSafe(mat, row, col, num):
```

```
            mat[row][col] = num
```

```
            if solveSudokuRec(mat, row, col + 1):
```

```
                return True
```

```
            mat[row][col] = 0
```

```
return False
```

```
def solveSudoku(mat):  
    solveSudokuRec(mat, 0, 0)
```

```
if __name__ == "__main__":
```

```
    mat = [  
        [3, 0, 6, 5, 0, 8, 4, 0, 0],  
        [5, 2, 0, 0, 0, 0, 0, 0, 0],  
        [0, 8, 7, 0, 0, 0, 0, 3, 1],  
        [0, 0, 3, 0, 1, 0, 0, 8, 0],  
        [9, 0, 0, 8, 6, 3, 0, 0, 5],  
        [0, 5, 0, 0, 9, 0, 6, 0, 0],  
        [1, 3, 0, 0, 0, 0, 2, 5, 0],  
        [0, 0, 0, 0, 0, 0, 0, 7, 4],  
        [0, 0, 5, 2, 0, 6, 3, 0, 0]  
    ]
```

```
    solveSudoku(mat)
```

```
    for row in mat:
```

```
        print(" ".join(map(str, row)))
```

Result :

... 3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

